O que falta (fonte única) — backlog operacional e critérios de "v1 vendável"

- **Última atualização**: Fev/2026 (05/02)

Este documento é a **fonte de verdade única** do "o que falta" a partir do estado atual do repositório (Fev/2026).
Ele substitui a visão antiga "capability-driven vs benchmarks" como lista operacional de execução.

> Referências: `docs/ROADMAP_3A5_ANOS.md` (Now/Next/Later) e `docs/O_QUE_FALTA_GAPS_VS_BENCHMARKS.md` (histórico/benchmark).

---

1) Estado atual (Fev/2026) — o que já foi entregue (resumo)

Core (eventos + execução)

- **Event Model canônico v1**: `api/events/eventModelV1.js`
- **Event ledger append-only (`event_ledger`)**: migration `api/db/migrations/20260204020000-create`
  + `api/services/eventLedgerService.js`
- **Retenção mínima (prune)**: `api/services/pruneEventLedgerService.js`
  (por `EVENT_LEDGER_RETENTION_DAYS`)
- **Execution Queue + Scheduler v1 (DB-first)**:
  - migrations `api/db/migrations/20260204021000-create-exec-jobs-and-schedules.js`
  - worker `api/services/executionQueueService.js`
  - scheduler `api/services/schedulerService.js`
  - SLA scan `api/services/opsSlaService.js` (`DeliverySLARisk`, `PhysicalSLARisk`)
- **Execution ledger por nó (v1, DB-first)**:
  - migrations `api/db/migrations/20260204031000-create-flow-execution-ledger.js`
    (tabelas `flow_executions` + `node_executions`)
  - serviço: `api/services/flowExecutionLedgerService.js` (header
    + steps por node; replay controlado em modo simulation)
- **Dispatcher de triggers a partir do `event_ledger` (v1 parcial)**:
  - cursor: migration `api/db/migrations/20260204033000-event-dispatch-state-and-schedule.j`
    (`event_dispatch_state` + schedule `scan_event_triggers`)
  - serviço: `api/services/eventLedgerTriggerDispatcher.js` (scan
    + enqueue + dispatch via webhooks)
  - extensão do estado: migration `api/db/migrations/20260204034000-event-dispatch-state-add`
  - trigger de flows (best-effort): `api/services/flowEventTriggerService.js`
    (mapeamento via `metadata.eventTriggers`)

Suites verticais (MVP→v1 operacional)

- **Fast service (filas físicas)**:
  - público por token: `/physical/*` (kiosk/panel/station) em `api/server.js`
  - operações adicionais (station): `start/skip/transfer`
  - admin: `GET/POST /api/physical/queues`, `GET /api/physical/metrics`
  - sequenciamento robusto: `physical_ticket_sequences` (migration `20260204022000-physical-queues-multiunit-and-sequences.js`)
- **Pedidos/Storefront/Delivery**:
  - público por token: `POST /storefront/orders`
  - hardening v1: idempotência por `idempotencyKey` (storefront) + propagação de `correlationId` (best-effort)
  - métricas: `GET /api/orders/metrics` (agregações por `locationId`)
  - incidente mínimo: `POST /orders/:id/incidents` (abre ticket digital e emite evento)
  - incidente operacional (omnichannel): ticket 1:1 com o pedido, idempotência e status
- **Omnichannel v1**:
  - unificação conversa ticket 1:1 com idempotência
  - status por mensagem (delivered/read/failed) na operação
  - roteamento/skills básicos, SLA simples e histórico unificado
- **Templates "solutions" por segmento**:
  - pacotes E2E (fastfood/pizzaria/clínica) com operação e métricas mínimas
- **PBX v1**:
  - base produto/realtime/CDR via migrations PBX
  - `dialplan/apply` com trilha mínima em `pbx_config_versions` (signature)
  - **Provisionamento de ramais (MVP)**: `POST /api/pbx/extensions` aceita `provisioning` (vendor+mac) e expõe `GET /provision/<tenantKey>/<vendor>/<mac>` (ou com token). Ver `docs/PBX_PROVISIONING_GRANDSTREAM_AVAYA.md`.
  - **Ops v1.1**: rotação de token e provisionamento em lote no admin (frontend).
  - **Call events v0 (estado + schedule)**: migration `api/db/migrations/20260204035000-pbx-call` + `api/services/pbx/pbxCallEventsService.js`.

Marketing (v1 completo)

- **Migrations e modelos base**: `api/db/migrations/20260204040000-marketing-v1-core.js` + models `Marketing*` em `api/db/models/`.
- **Serviço inicial**: `api/services/marketing/marketingService.js`.
- **Público**: `GET /mkt/track.js`, `POST /mkt/e`, `GET /lp/:slug`, `POST /lp/:slug/submit`.
- **Campanhas de email**: `POST /api/marketing/email-campaigns/:id/send` (envio best-effort).

- **UI/admin completa**: gestão de forms/leads/scoring/email + jornadas v1 e funil visual básico.

Voz (base técnica entregue)

- **Voice nodes v0 (engine)**: `engine/nodes/voiceNodes.js` + `engine/__tests__/voiceNodes.test.js`.
- **Ingestão básica de eventos de voz**: `api/services/voiceEventService.js` + `POST /api/voice/events`.

RPA Flow Pipeline (base entregue)

- **Serviço**: `api/services/rpaFlowPipelineService.js` (análise → seeds → validação → conversão flow→RPA).
- **Endpoints**: preview e execução no `api/server.js`.
- **Playbook operacional**: `docs/RPA_FLOW_PIPELINE_PLAYBOOK.md`.

SaaS e governança (base enterprise já entregue)

- **Multi-tenant + RBAC + planos/limites + auditoria + super-admin + onboarding + rate-limit por tenant**: ver `IMPLEMENTACAO_SAAS_COMPLETA.md`.

Hardening de IA (contratos + safe-mode)

- **AI Output Envelope (obrigatório)**:
  - especificação: `docs/IA_OUTPUT_ENVELOPE.md`
  - schema: `contracts/ia/ai-output-envelope.schema.json`
  - implementação/testes: `engine/ai/aiEnvelope.js` e `tests/unit/aiEnvelope.test.js`

Integrações de plataforma (base)

- **Webhooks customizados**: `api/services/webhookService.js`
- **WebSockets (real-time por tenant)**: `api/services/websocketService.js`
- **Validação de inputs**: `api/utils/inputValidator.js` (resumo em `IMPLEMENTACOES_MELHORIAS.md`)

Analytics e marketplace (primeiro corte)

- **Analytics/ROI v0**: `api/services/roiService.js` + `GET /analytics/roi` + UI em `frontend/src/pages/AnalyticsPage.jsx`.
- **Node marketplace v0** (catálogo estático): `api/services/nodeMarketplaceService.js` + `GET /api/node-marketplace/catalog` + UI `frontend/src/pages/NodeMarketplacePage.jsx`.
  - **Próximo foco**: evoluir para **Analytics/ROI v1** (funil básico, custos por canal e KPIs de SLA).

Governança enterprise (primeiro corte)

- **Ambientes de flow (dev/stage/prod)**: `GET /flows?environment=...` + `PUT /flows/:id/environment` + seletor no editor.
- **SSO/SCIM (config por tenant)**: `api/services/tenantSettingsService.js` + endpoints `/api/tenant-settings/:key` + UI `frontend/src/pages/SsoScimPage.jsx`.
- **Backup/DR básico**: `GET /api/backup/export` + `POST /api/backup/import` + UI `frontend/src/pages/BackupPage.jsx`.

Gate operacional

- **Selftest**: `GET /health/selftest`
- **Runner de stack**: `scripts/system-autotest-stack.sh` com gate de selftest (FAIL bloqueia)
- **Unit tests**: `npm run test:unit` passando no estado atual
- **Bootstrap/autoprovisionamento (base)**:
  - `startInitialization()` em `api/server.js` garante `systemadmin`/tenant `system` (primeira inicialização).
  - Seed idempotente: `api/db/seed.js` (+ utilitário `scripts/create-super-admin.js`).
  - One-click bootstrap (super-admin): `POST /super-admin/bootstrap/one-click` (serviço `api/services/oneClickBootstrapService.js`).

Entregas concluídas (ex-backlog)

- **Execution ledger**: endpoints de consulta (execution_id/correlation_id/subject_id) com RBAC e replay parcial "from node" com trilha auditável.
- **Queue/Scheduler enterprise mínimo**: quotas por tenant, DLQ consultável/reprocessável, métricas (lag/falhas/retries) e selftest por limiar.
- **Triggers do `event_ledger` (v1)**: mapping evento→flow/trigger com idempotência; dispatcher v1 para webhooks e disparo best-effort de flows.
- **PBX call events v0**: ringing/answered/ended no `event_ledger`, correlação com ticket/pedido e base DB-first (estado + schedule).
- **Fast service v1**: auditoria operacional consultável, métricas completas e contratos de estados com idempotência.
- **Orders/Delivery v1**: KPIs `order_to_ready`/`order_to_deliver` + alertas e dashboard mínimo.
- **Tokens públicos**: endpoints admin (listar/revogar/rotacionar), rate-limit por token/kind e dashboards mínimos de uso.
- **RPA Flow Pipeline v1.1**: documentação/playbooks, testes E2E e validações de segurança, observabilidade (logs/métricas/audit).
- **Autoprovisionamento v1**: checklist/relatório por step, opções por suíte/unidade e gates de segurança.
- **PBX provisioning v1.1**: templates vendor/model/firmware, parâmetros operacionais, rotação/expiração de token com trilha auditável e bulk + sanity check.
- **Core DAG + event model (consolidação)**: triggers/handlers full-fidelity do `event_ledger` e retenção por tenant com custo previsível.

- **Plataforma integrada**: SDK/marketplace de nodes, Analytics/ROI completo e Voz v1 no DAG.
- **Enterprise**: HA/DR, auditoria imutável, programa de parceiros e marketplace monetizado.

---

2) Backlog único (priorizado) — Now/Next/Later (o que falta)

> Regra: cada item abaixo é "único" (não duplicar em outros docs). Se um tema aparecer em mais de um pilar, ele deve ser referenciado (não replicado).

Now (0–3 meses) — hardening para v1 vendável (suíte operável)

2.1 Execução "enterprise mínima" (core DAG)

- **Conectar o execution ledger por nó ao runtime + expor consulta/replay assistido**
  - **Status**: base DB-first já existe (`flow_executions`/`node_executions` + `flowExecutionLedgerService`); integração full do runtime ainda parcial.
  - **Saídas mínimas**:
    - registrar execuções reais do DAG (não só simulation/flows específicos)
  - **Critério de pronto**: responder "o que aconteceu em qual node" e permitir replay controlado com rastreabilidade.

2.2 Triggers do runtime a partir do `event_ledger`

- **Consumer/dispatcher** de `event_ledger` para acionar execuções (regras mínimas)
  - retry/backoff e DLQ integrados ao runtime/queue
  - falta conexão completa com o runtime do DAG (idempotência/observabilidade/DLQ).

2.3 PBX: integração de call events v0 ao runtime

- acionar triggers/ações do DAG a partir dos eventos de chamada (ringing/answered/ended)
- garantir observabilidade e correlação ponta a ponta em produção

Next (3–6 meses) — produto inicial por suíte + analytics/ROI v1

2.4 Analytics/ROI v1 (sair do v0)

- funil visual básico e atribuição mínima por canal

- custos por canal + KPIs de SLA mínimos (order_to_ready/order_to_deliver)
- dashboards operacionais por suíte (fast service + orders/delivery)

Next (6–12 meses) — consolidação core + PBX/voz v1

2.5 PBX + voz v1 (base CCaaS)

- URA visual v1, filas de voz e gravação com trilha auditável
- observabilidade de voz (CDR/eventos) e custos básicos

---

3) Gates e "Definição de Pronto" (DoD) — para declarar v1 vendável

3.1 Gate mínimo de CI/Stack

- `scripts/system-autotest-stack.sh` deve cobrir (no mínimo):
  - token público → pedido → transições (kitchen/dispatch) → status público
  - token público → kiosk cria ticket → station opera (call/start/finish/skip/transfer) → panel reflete
  - `/health/selftest` retorna `PASS` ou `WARN` (FAIL bloqueia)
- Gate oficial: `tools/system/validate.js` deve passar (contratos + QA completa).
- Autotest: `scripts/autotest.js` deve resultar em veredito `GO`.
  - **Status**: `tools/system/validate.js` passou localmente em 05/02 (18/18 RPA); `scripts/autotest.js` retornou `GO` (sem Docker).

3.2 DoD (por tema)

- **Ledger por nó + replay**: execução auditável (inputs/outputs), replay parcial controlado, retenção por tenant.
- **Queue/scheduler enterprise mínimo**: quotas, DLQ, métricas e self-test por limiar.
- **Suites vendáveis** (Fast service + Orders/Delivery): estados consistentes + idempotência + auditoria + KPIs mínimos.